

Санкт-Петербургский политехнический университет Петра Великого
Институт прикладной математики и механики
Высшая школа прикладной математики и вычислительной физики

Параллельные вычисления

Лабораторная работа №1

«Решение задачи Коши для системы линейных неоднородных ОДУ
методом Пикара»

Работу выполнила:
студентка II курса
магистратуры
Добрецова Е.В.
Группа:
5040102/40101
Преподаватель:
Козлов К.Н.

Санкт-Петербург
2026

Содержание

1. Постановка задачи и её формализация	3
2. Алгоритм метода Пикара и условия его применимости	3
2.1. Описание метода	3
2.2. Параллелизация вычислений	4
2.3. Условия применимости метода	4
2.4. Стандарт OpenMP	4
3. Предварительный анализ задачи (проверка условий применимости метода)	5
4. Тестовый пример с детальными расчётами для задачи малой размерности	5
5. Визуализация решения и погрешности	9
6. Подготовка контрольных тестов для иллюстрации параллельной реализации метода	9
7. Модульная структура программы	10
8. Численный анализ решения задачи	12
9. Выводы	15
10. Примечания	15

1. Постановка задачи и её формализация

Дана система линейных неоднородных обыкновенных дифференциальных уравнений с постоянными коэффициентами:

$$\frac{d\mathbf{y}}{dt} = A\mathbf{y} + \mathbf{f}(t), \quad (1)$$

и следующими начальными условиями:

$$y_i = 1, \quad i = \overline{1, n}. \quad (2)$$

Здесь $\mathbf{y}(t) = \begin{pmatrix} y_1(t) \\ y_2(t) \\ \dots \\ y_n(t) \end{pmatrix}$, $\mathbf{f}(t) = \begin{pmatrix} f_1(t) \\ f_2(t) \\ \dots \\ f_n(t) \end{pmatrix}$, матрица A имеет размерность $n \times n$ и является верхнетреугольной ($a_{ij} = 0$ при $i > j$) с единичными ненулевыми элементами.

Необходимо решить задачу методом последовательных приближений Пикара с применением циклической схемы распределения уравнений по процессам для обеспечения равномерной загрузки процессоров, осуществляя программирование на языке C с применением технологии OpenMP. Также требуется провести анализ метода путём следующих исследований:

- зависимость времени выполнения от размера системы;
- зависимость времени выполнения от числа параллельных процессов/потоков;
- зависимость ускорения от числа параллельных процессов/потоков;
- зависимость эффективности параллелизации от числа параллельных процессов/потоков.

2. Алгоритм метода Пикара и условия его применимости

2.1. Описание метода

Метод Пикара для задачи (1) с начальными условиями (2) определяет последовательность приближений $\mathbf{y}^{(k)}(t)$ следующим образом:

$$\mathbf{y}^{(k+1)}(t) = \mathbf{y}_0 + \int_{t_0}^t (A\mathbf{y}^{(k)}(\tau) + \mathbf{f}(\tau)) d\tau, \quad k = 0, 1, 2, \dots$$

Процесс продолжается до выполнения критерия сходимости:

$$\|\mathbf{y}^{(k+1)} - \mathbf{y}^{(k)}\| < \varepsilon$$

Интеграл вычисляется численно по квадратурной формуле трапеций на каждом временном интервале $[t_m, t_{m+1}]$:

$$\mathbf{y}_{m+1}^{(k+1)} = \mathbf{y}_0 + \frac{\Delta t}{2} \left[A\mathbf{y}_m^{(k)} + \mathbf{f}(t_m) + A\mathbf{y}_{m+1}^{(k)} + \mathbf{f}(t_{m+1}) \right], \quad (3)$$

где $\Delta t = t_{m+1} - t_m$, $\mathbf{y}_m^{(k)}$ - приближение в узле t_m на k -й итерации.

2.2. Параллелизация вычислений

На каждой итерации k новое приближение решения $y^{(k+1)}(t)$ вычисляется по формуле (3) и зависит только от значений предыдущей итерации $y^{(k)}(t)$, что позволяет вычислять значения $y^{(k+1)}(t)$ в различных временных узлах независимо друг от друга в рамках одной итерации. Вычисления на отдельных временных слоях распределяются между потоками с использованием директив стандарта OpenMP.

2.3. Условия применимости метода

Итерационный процесс Пикара сходится при выполнении следующих условий:

1. липшицевость правой части по переменной y ;
2. ограниченность интервала интегрирования;
3. корректность задания начальных условий.

2.4. Стандарт OpenMP

OpenMP (от англ. Open Multi-Processing) - открытый стандарт для распараллеливания программ на языках C, C++ и Fortran, дающий описание совокупности директив компилятора, библиотечных процедур и переменных окружения, предназначенных для программирования многопоточных приложений на многопроцессорных системах с общей памятью.

3. Предварительный анализ задачи (проверка условий применимости метода)

Проверим условие Липшица по y для правой части в рассматриваемой области.

$F(t, y) = Ay + f(t)$ линейна по y , поэтому удовлетворяет усл. Липшица в любой ограниченной области для любых y_1, y_2 :

$$\|F(t, y_1) - F(t, y_2)\| = \|A(y_1 - y_2)\| \leq L \|y_1 - y_2\|, \\ L = \|A\|$$

Для обеспечения сходимости метода Тихара отрезок выбирается из условия сжимаемости оператора. Поскольку правая часть удовлетворяет условию Липшица с константой $L = \|A\|$, а левая граница принята за 0, правая граница t_m должна быть выбрана с учётом ограничения на длину отрезка:

$$t_m \leq \frac{1}{\|A\|}$$

$$\|A\| \leq Cn, \text{ где } C = \text{const.}$$

Таким образом, для сходимости метода Тихара правая граница отрезка вычисляется по формуле

$$t_m = \frac{1}{C(n+1)}.$$

В данной задаче $C = 250$.

4. Тестовый пример с детальными расчётами для задачи малой размерности

Дана система из трех линейных неодн. ОДУ:

$$\frac{dy}{dt} = Ay + f(t), \quad y(t) \in \mathbb{R}^3$$

$$A = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix}, \quad f(t) = \begin{pmatrix} \sin t \\ \sin(t+1) \\ \sin(t+2) \end{pmatrix}, \quad 0 \leq t \leq 0,3, \quad \Delta t = 0,1$$

Сетка: $t_m = \frac{m}{10}, m = \overline{0,3}$

Начальные условия: $y_m^{(0)} = 1, m = \overline{0,3}$

Дальнейшие значения $y_{m+1}^{(k+1)}$ находятся по формуле (3).

Итерация 1: $Ay_0^{(0)} = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 \\ 2 \\ 1 \end{pmatrix} = Ay_1^{(0)}$

$$f(t_0) = \begin{pmatrix} \sin 0 \\ \sin 1 \\ \sin 2 \end{pmatrix} \approx \begin{pmatrix} 0 \\ 0,841 \\ 0,909 \end{pmatrix}$$

$$f(t_1) = \begin{pmatrix} \sin 0,1 \\ \sin 1,1 \\ \sin 2,1 \end{pmatrix} \approx \begin{pmatrix} 0,0998 \\ 0,891 \\ 0,863 \end{pmatrix}$$

По формуле (3):

$$y_1^{(1)} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} + 0,05 \left(\begin{pmatrix} 3 \\ 2 \\ 1 \end{pmatrix} + \begin{pmatrix} 0 \\ 0,841 \\ 0,909 \end{pmatrix} + \begin{pmatrix} 3 \\ 2 \\ 1 \end{pmatrix} + \begin{pmatrix} 0,0998 \\ 0,891 \\ 0,863 \end{pmatrix} \right) =$$

$$= \begin{pmatrix} 1,305 \\ 1,287 \\ 1,189 \end{pmatrix}$$

$$f(t_2) = \begin{pmatrix} \sin 0,2 \\ \sin 1,2 \\ \sin 2,2 \end{pmatrix} = \begin{pmatrix} 0,199 \\ 0,932 \\ 0,808 \end{pmatrix}$$

$$y_2^{(1)} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} + 0,05 \left(\begin{pmatrix} 3 \\ 2 \\ 1 \end{pmatrix} + \begin{pmatrix} 0,199 \\ 0,932 \\ 0,808 \end{pmatrix} + \begin{pmatrix} 3 \\ 2 \\ 1 \end{pmatrix} + \begin{pmatrix} 0,0998 \\ 0,891 \\ 0,863 \end{pmatrix} \right) =$$

$$= \begin{pmatrix} 1,315 \\ 1,291 \\ 1,184 \end{pmatrix}$$

$$f(t_3) = \begin{pmatrix} \sin 0,3 \\ \sin 1,3 \\ \sin 2,3 \end{pmatrix} = \begin{pmatrix} 0,296 \\ 0,964 \\ 0,746 \end{pmatrix}$$

$$y_3^{(1)} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} + 0,05 \left(\begin{pmatrix} 3 \\ 2 \\ 1 \end{pmatrix} + \begin{pmatrix} 0,296 \\ 0,964 \\ 0,746 \end{pmatrix} + \begin{pmatrix} 3 \\ 2 \\ 1 \end{pmatrix} + \begin{pmatrix} 0,199 \\ 0,932 \\ 0,808 \end{pmatrix} \right) =$$

$$= \begin{pmatrix} 1,325 \\ 1,295 \\ 1,178 \end{pmatrix}$$

$$y_0^{(1)} = (1, 1, 1)^T$$

$$y_1^{(1)} = (1,305; 1,287; 1,189)^T$$

$$y_2^{(1)} = (1,315; 1,291; 1,184)^T$$

$$y_3^{(1)} = (1,325; 1,295; 1,178)^T$$

Упражнение 2

$$y_0^{(2)} = (1, 1, 1)^T$$

$$y_1^{(2)} = y_0 + 0,05(Ay_0^{(1)} + f(t_0) + Ay_1^{(1)} + f(t_1))$$

$$Ay_0^{(1)} = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 \\ 2 \\ 1 \end{pmatrix}$$

$$Ay_1^{(1)} = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1,305 \\ 1,287 \\ 1,189 \end{pmatrix} = \begin{pmatrix} 3,781 \\ 2,476 \\ 1,189 \end{pmatrix}$$

$$y_1^{(2)} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} + 0,05 \left(\begin{pmatrix} 3 \\ 2 \\ 1 \end{pmatrix} + \begin{pmatrix} 0 \\ 0,841 \\ 0,909 \end{pmatrix} + \begin{pmatrix} 3,781 \\ 2,476 \\ 1,189 \end{pmatrix} + \begin{pmatrix} 0,0998 \\ 0,891 \\ 0,863 \end{pmatrix} \right) =$$

$$= \begin{pmatrix} 1,344 \\ 1,310 \\ 1,198 \end{pmatrix}$$

$$Ay_2^{(1)} = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1,315 \\ 1,291 \\ 1,184 \end{pmatrix} = \begin{pmatrix} 3,790 \\ 2,475 \\ 1,184 \end{pmatrix}$$

$$y_2^{(2)} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} + 0,05 \left(\begin{pmatrix} 3,781 \\ 2,476 \\ 1,189 \end{pmatrix} + \begin{pmatrix} 0,0998 \\ 0,891 \\ 0,863 \end{pmatrix} + \begin{pmatrix} 3,790 \\ 2,475 \\ 1,184 \end{pmatrix} + \begin{pmatrix} 0,199 \\ 0,932 \\ 0,808 \end{pmatrix} \right) =$$

$$= \begin{pmatrix} 1,393 \\ 1,339 \\ 1,202 \end{pmatrix}$$

$$Ay_3^{(1)} = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1,325 \\ 1,295 \\ 1,178 \end{pmatrix} = \begin{pmatrix} 3,798 \\ 2,473 \\ 1,178 \end{pmatrix}$$

$$y_3^{(2)} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} + \left(\begin{pmatrix} 3,790 \\ 2,475 \\ 1,184 \end{pmatrix} + \begin{pmatrix} 0,199 \\ 0,932 \\ 0,808 \end{pmatrix} + \begin{pmatrix} 3,798 \\ 2,473 \\ 1,178 \end{pmatrix} + \begin{pmatrix} 0,296 \\ 0,964 \\ 0,746 \end{pmatrix} \right) =$$

$$= \begin{pmatrix} 1,404 \\ 1,342 \\ 1,196 \end{pmatrix}$$

$$y_0^{(2)} = (1, 1, 1)^T$$

$$y_1^{(2)} = (1,344; 1,310; 1,198)^T$$

$$y_2^{(2)} = (1,393; 1,339; 1,202)^T$$

$$y_3^{(2)} = (1,404; 1,342; 1,196)^T$$

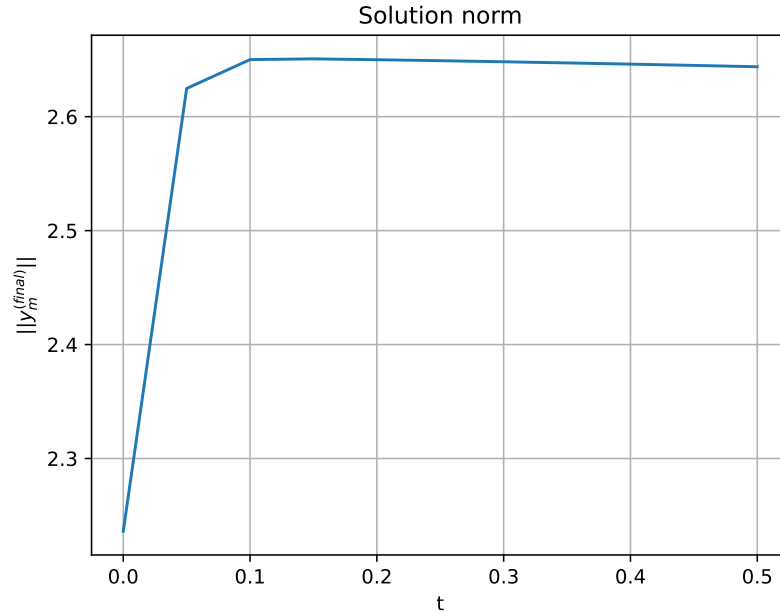


Рис. 5.1. Зависимость нормы вектора решения в узлах от времени

5. Визуализация решения и погрешности

Зададим размерность системы $n = 5$, $m = 10$.

В качестве визуализации решения задачи будем строить график нормы вектора решения в узлах от времени t . График можно видеть на Рис. 5.1. Отметим, что характер зависимости является следствием особенности задачи и выбранной правой части.

Также построим график зависимости нормы разности текущего и предыдущего приближений от номера итераций. Будем использовать логарифмический масштаб по оси y . Данный график приведён на Рис. 5.2.

6. Подготовка контрольных тестов для иллюстрации параллельной реализации метода

Во всех исследованиях правая часть СЛАУ имеет вид $\mathbf{f}(t) = \sin(t + i)$, $i = \overline{(0, n - 1)}$.

Для исследования зависимости времени выполнения программы от размера системы зафиксируем количество потоков $p = 4$, а также количество узлов $m = 1000$. Будем поочерёдно запускать метод на системах размерности от 1 до 30.

В исследованиях времени выполнения, ускорения и эффективности параллелизации от количества параллельных процессов будем брать поочерёдно значения количества потоков p от 1 до 12, количество потоков ограничивается количеством ядер процессора,

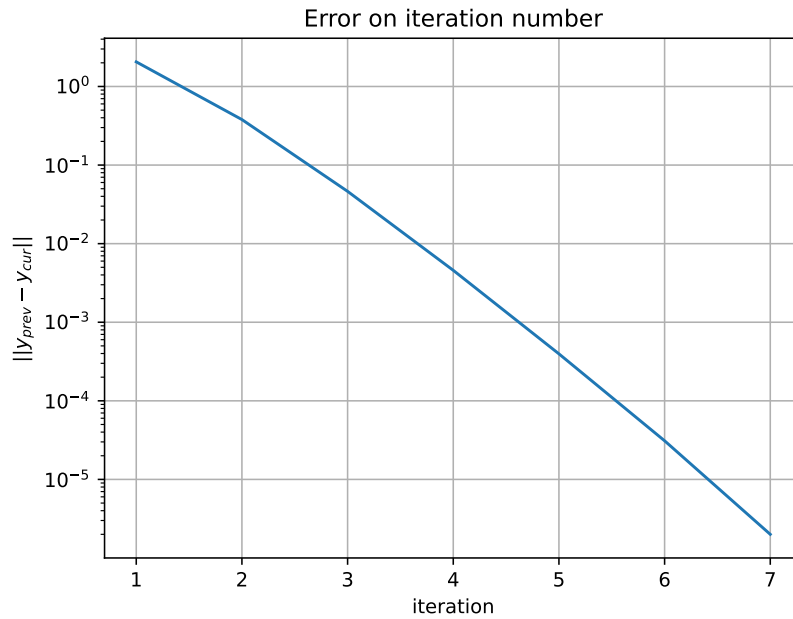


Рис. 5.2. Зависимость нормы ошибки от номера итерации

равным 6. Зафиксируем размерность системы $n = 1000$ и количество узлов $m = 3500$, данные значения обеспечивают достаточную вычислительную нагрузку для проявления эффекта параллелизации.

Ускорение находится по формуле $a = \frac{T_1}{T_p}$, где T_1 - время выполнения программы без параллелизации, T_p - время выполнения при числе потоков p . Эффективность параллелизации вычисляется по формуле $E = \frac{T_1}{p \cdot T_p}$.

7. Модульная структура программы

```

picard├─
  analysis├─
    └─ plots.ipynb├─
  results├─
  src├─
    CMakeLists.txt├─
    generate.c├─
    generate.h├─
    main.c├─
    matrix_tools.c├─
    matrix_tools.h├─
    picard.c├─
    picard.h├─
    study.c├─

```

Приведём на **листинге** дерево проекта.

В файле `analysis/plots.ipynb` содержится код чтения данных из `.csv` файлов и построение графиков средствами `matplotlib.pyplot`.

Опишем файлы из директории `src`, где содержится C-код метода.

- `matrix_tools.c`

- `double **alloc_matrix(int n, int m)` - выделение памяти для матрицы размера $n \times m$;
- `void free_matrix(double **y, int n)` - освобождение памяти, занятой массивом из n массивов;
- `double compute_diff_norm(int n, int m, double **y1, double **y2)` - вычисление строковой максимум-нормы разности матриц;
- `void matrix_mul_vector(int n, double** A, double* y_m, double* res)` - умножение матрицы на вектор.

- `picard.c`

- `double **picard_method(int n, double **f, double**A, double eps, double t_0, double t, int m)` - функция метода Пикара для задачи размерности n с m узлами, матрицей A и вектором правой части f на отрезке $[t_0, t]$ с точностью eps .

- `generate.c`

- `double **generate_rhs(int n, int m, double t_0, double t)` - генерация правой части для контрольного примера;
- `int generate_threads(int *threads, int max_threads)` - генерация последовательности чисел от 1 до `max_threads` и её сохранение;
- `double **generate_matrix(int n)` - генерация матрицы для контрольного примера согласно **постановке задачи**.

- `study.c`

- `void study_time_vs_size(void)` - изучение зависимости времени выполнения программы от разных значений размерности системы. Функция перебирает значения размерности от 1 до 30 при фиксированном количестве потоков и записывает время выполнения программы для каждого значения размерности в `.csv`-файл для построения графика;

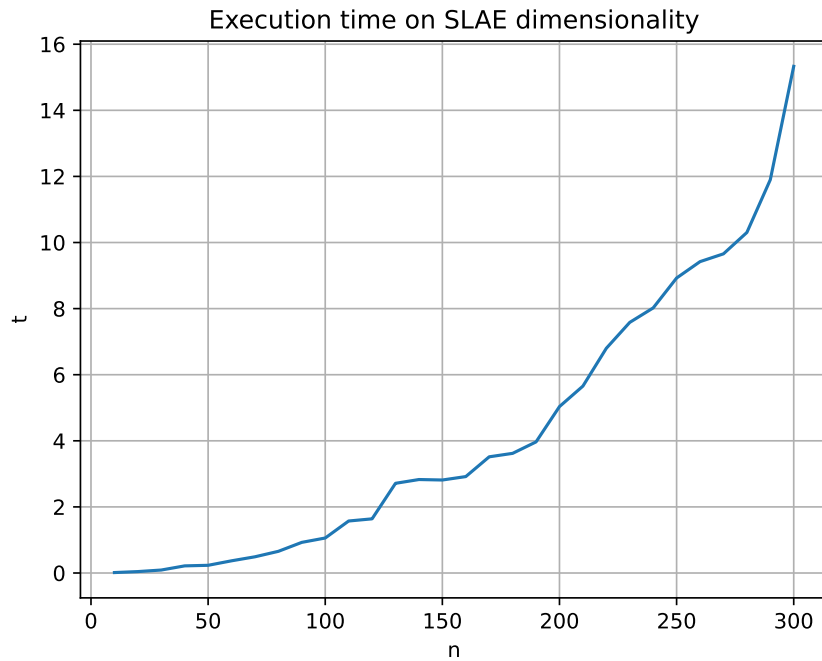


Рис. 8.1. Зависимость времени исполнения программы от размерности системы при 4 потоках

- `void study_dependencies_on_threads(void)` - изучение зависимости времени выполнения программы, ускорения и эффективности параллелизации от числа параллельных процессов. Функция фиксирует размерность системы и число узлов и решает задачу методом Пикара для каждого значения числа параллельных процессов, вычисляя все перечисленные выше величины и записывая их в `.csv`-файл.

- `main.c`

- `void run_test_case(void)` - функция исполнения **тестового примера** для проверки корректности реализации метода;
- `int main(void)` - запуск функций из `study.c`.

8. Численный анализ решения задачи

Значения фиксированных переменных и диапазоны меняющихся для описанных ниже исследований приведены **выше**.

На **Рис. 8.1** показан график зависимости времени выполнения программы от размерности системы. Из графика видно, что время выполнения программы с ростом размерности системы возрастает квадратично, что обусловлено квадратичной вычислительной сложностью операций умножения матрицы на вектор на каждом шаге метода Пикара.

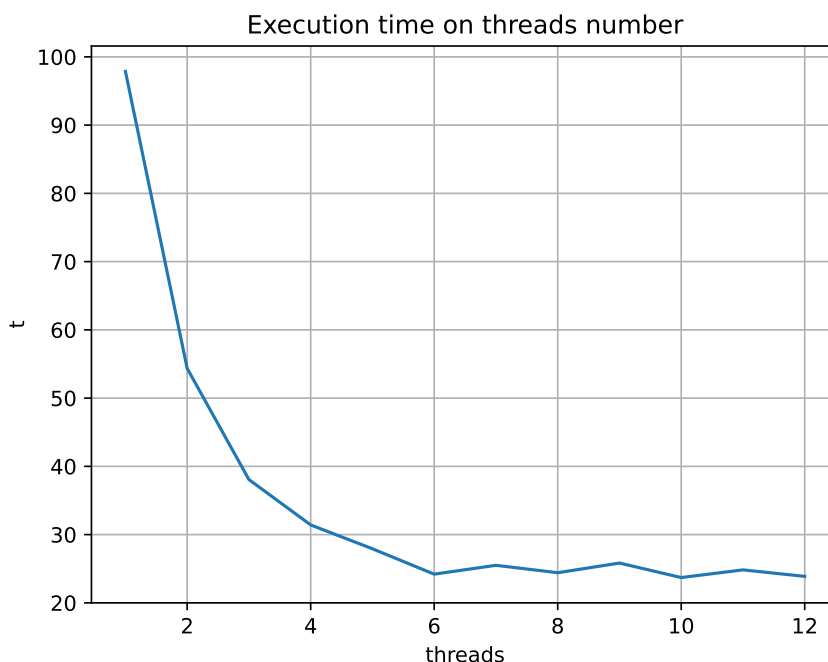


Рис. 8.2. Зависимость времени исполнения программы от числа параллельных процессов

Параллельная реализация с использованием четырёх потоков позволяет сократить абсолютное время выполнения программы, но не меняет характер зависимости. Небольшие неровности на графике связаны с особенностями работы с памятью и накладными расходами параллельных вычислений.

График зависимости времени выполнения программы от числа параллельных процессов показан на [Рис. 8.2](#).

Согласно графику, при увеличении числа потоков от 1 до 4-5 наблюдается существенное сокращение времени исполнения программы. При дальнейшем увеличении числа потоков снижение времени замедляется и, более того, возможно нго увеличение. Это связано с ростом накладных расходов на синхронизацию потоков и работу с памятью.

На [Рис. 8.3](#) представлен график ускорения в зависимости от числа параллельных процессов/потоков. Ускорение растёт практически линейно при числе потоков до 4-5, максимальное ускорение составляет около 4.1-4.2 и достигается при 11 потоках. Нелинейный характер роста ускорения и наличие локальных провалов обусловлены накладными расходами OpenMP и конкуренцией потоков за доступ к памяти.

График эффективности параллелизации от числа потоков приведён на [Рис. 8.4](#). Можно видеть, что при увеличении числа потоков накладные расходы и ограничения архитектуры начинают доминировать, в связи с этим жффективность параллелизации с ростом числа потоков снижается. Однако при малом числе потоков наблюдается небольшой рост эффективности.

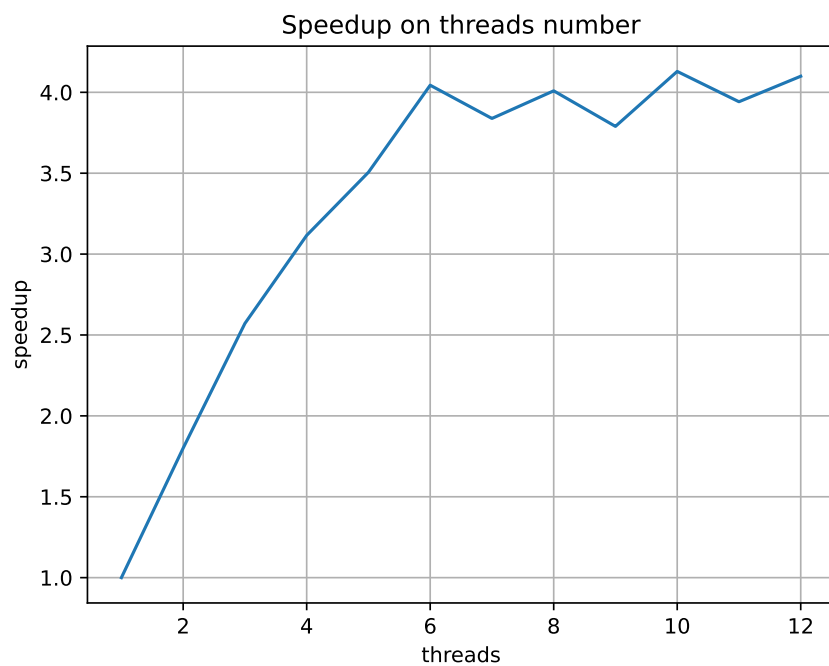


Рис. 8.3. Зависимость ускорения от числа параллельных процессов

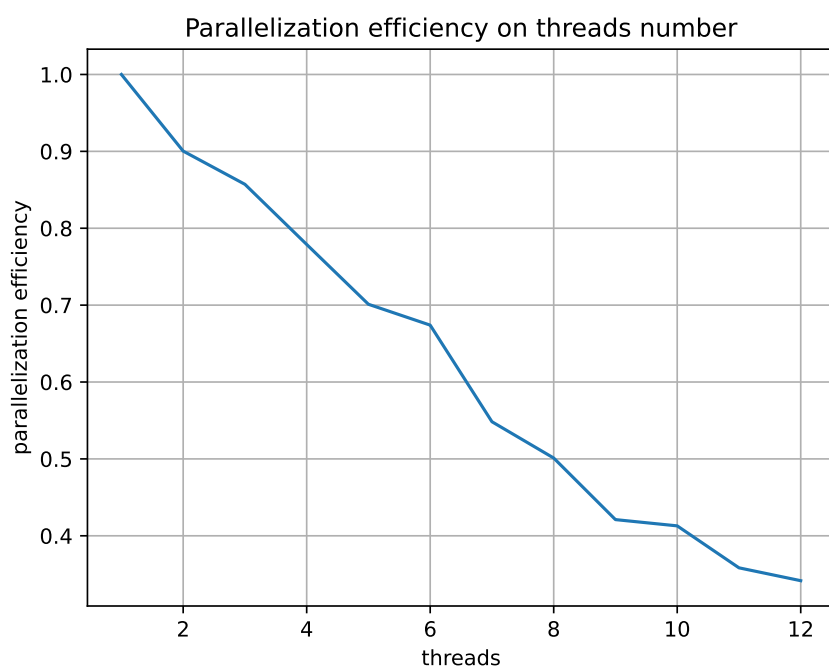


Рис. 8.4. Зависимость эффективности параллелизации от числа параллельных процессов

9. Выводы

В ходе выполнения данной работы была реализована параллельная версия метода Пикара с применением циклической схемы распределения уравнений по процессам с использованием технологии OpenMP, а также проведён ряд вычислительных экспериментов.

Исследования показали, что с ростом размерности системы время выполнения программы возрастает квадратично. Такой характер роста обусловлен квадратичной вычислительной сложностью операции умножения матрицы на вектор.

При увеличении числа потоков наблюдается существенное сокращение времени выполнения программы по сравнению с её последовательным исполнением. Наибольший выигрыш достигается при увеличении числа потоков до значения, близкого к числу вычислительных ядер компьютера. Дальнейшее увеличение числа потоков не приводит к заметному сокращению исполняемого времени вследствие роста накладных расходов и ограничений архитектуры памяти.

Анализ ускорения показал, что при малом числе потоков наблюдается близкий к линейному рост ускорения, однако при числе потоков, большем 5, ускорение практически останавливается.

Эффективность параллелизации незначительно растёт при малом числе потоков, но при его дальнейшем увеличении эффективность снижается вследствие роста накладных расходов на синхронизацию и конкуренции за ресурсы.

Таким образом, параллельная реализация метода Пикара позволяет существенно сократить время исполнения для задач большой размерности.

10. Примечания

Код реализации метода доступен [здесь](#).